

QBUS 2820: Predictive Analytics, S2 2026

Week 3: K-Nearest Neighbours

Dr. Qin Fang

Discipline of Business Analytics, The University of Sydney Business School

K-Nearest Neighbours

1. K-Nearest Neighbours (KNN)
2. KNN properties
3. Comparison with linear regression
4. Summary

Reading: Chapter 3.5 of ISL.

Exercise questions: Chapter 3.7 of ISL, Q1, Q3 and Q4.

Introduction

In our first lecture, we distinguished between parametric and nonparametric methods.

- Parametric methods assume a parametric form for the underlying model that generated the data. The underlying model that generated the data is described by a functional form that depends on a vector of unknown parameters θ .
- E.g., simple linear regression

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma^2)$$

is a parametric model as we assume the model that generated data y_i , given x_i , is normal distribution with mean $\beta_0 + \beta_1 x_i$ and variance σ^2 . The set of unknown parameters is $\theta = (\beta_0, \beta_1, \sigma^2)$.

Introduction

- Nonparametric methods do not assume a functional form for the underlying model that generated the data.
- Hence, nonparametric methods are more robust. But they have limitations too ... (discussed later)

Introduction

In this lecture, we consider the K-nearest neighbours method, a simple nonparametric approach.

Studying this method will also allow you to learn and consolidate important concepts in statistical learning.

For example, KNN can be considered as a special case of kernel methods - a set of popular non-parametric techniques.

K-Nearest Neighbours (KNN)

K-nearest neighbours

- K-nearest neighbours can be used for regression or classification. This week we will be focusing on regression.
- KNN requires no distribution assumptions.
- Mostly used as a supervised learning method

K-nearest neighbours

Problem: Given a training sample $\mathcal{D} = \{(y_i, \mathbf{x}_i)\}_{i=1}^N$. Find the prediction for the response variable Y at an input point $\mathbf{x}$.

The **K-nearest neighbours** (KNN) regression prediction of Y at input point $\mathbf{x}$ is defined as

$$\hat{f}(\mathbf{x}) = \frac{1}{k} \sum_{\mathbf{x}_i \in N_k(\mathbf{x}, \mathcal{D})} y_i,$$

where $N_k(\mathbf{x}, \mathcal{D})$ is the neighbourhood set containing k input points among $\{\mathbf{x}_i\}_{i=1}^N$ that are closest to $\mathbf{x}$.

Interpretation: the kNN prediction is the sample average of the k training response values whose corresponding inputs are closest to the query point $\mathbf{x}$.

Note about notation: kNN or KNN doesn't matter.

K-nearest neighbours

- KNN works well under the *implicit* continuity assumption that: if $\mathbf{x}_i$ is close to $\mathbf{x}_j$ then the corresponding optimal predictions (under the squared loss) $E(Y|X = \mathbf{x}_i)$ is close to $E(Y|X = \mathbf{x}_j)$
- Then, kNN approximates $E(Y|X = \mathbf{x})$ by an average of the response values among a neighborhood of $\mathbf{x}$.

Illustration

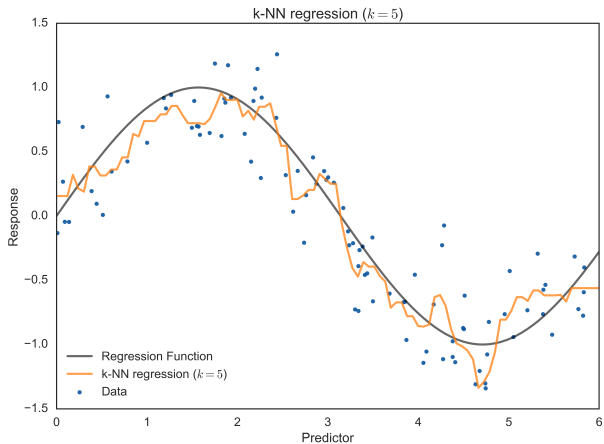
- We consider the regression model:

$$Y_i = \sin(x_i) + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2),$$

where we set $\sigma^2 = 0.1$ and restrict X to be between 0 and 6.

- In the next slide, we simulate $N = 100$ observations from the model and fit a KNN regression to this training sample.

Illustration ($k = 5$)



KNN properties

Modelling choices

$$\hat{f}(\mathbf{x}) = \frac{1}{k} \sum_{\mathbf{x}_i \in N_k(\mathbf{x}, \mathcal{D})} y_i,$$

for a training sample $\mathcal{D} = \{(y_i, \mathbf{x}_i)\}_{i=1}^N$.

The KNN method requires us to specify:

1. A distance metric.
2. The number of neighbours k .
3. The relevant/important predictors.

Distance

A common distance measure is the Euclidean distance:

$$d(\mathbf{x}_i, \mathbf{x}_l) = \sqrt{\sum_{j=1}^p (x_{ij} - x_{lj})^2}$$

We will often use the notation $\|\mathbf{x}_i - \mathbf{x}_l\|_2$, which denotes the ℓ_2 (Euclidean) norm of the vector $\mathbf{x}_i - \mathbf{x}_l$.

Other distances can also be used.

Mahalanobis distance

The Euclidean distance only makes sense if the predictors are in the same scale. An alternative is use the normalized Euclidean distance:

$$d(\mathbf{x}_i, \mathbf{x}_l) = \sqrt{\sum_{j=1}^p \left(\frac{x_{ij} - x_{lj}}{s_{x_j}} \right)^2},$$

where s_{x_j} is the sample standard deviation of predictor j in the training sample.

A more complicated measure that often works better in practice is the Mahalanobis distance

$$d(\mathbf{x}_i, \mathbf{x}_l) = \sqrt{(\mathbf{x}_i - \mathbf{x}_l)^T S^{-1} (\mathbf{x}_i - \mathbf{x}_l)},$$

where S^{-1} is the sample covariance matrix of the predictors.

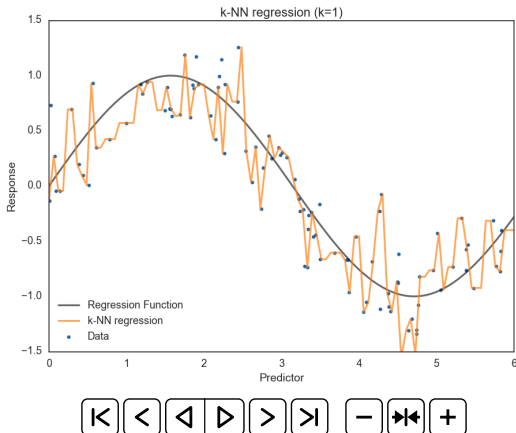
Choosing the number of neighbours

- The number of neighbours k is a **hyperparameter**: we need to specify it prior to the learning process.
- We cannot use the training data to pick k since we would then always choose $k = 1$ and fit the data perfectly!
- Instead, we select k based on bias-variance trade-off considerations.
- Choosing k is a model selection problem that will be considered in much detail in Weeks 4-6.

Overfitting and underfitting

- With a small k , the bias will be relatively small since the regression function evaluated at the neighbours $f(x_\ell)$ will be close to $f(x_0)$ (x_0 is prediction point). However, a small k means that we are averaging only a few observations, leading to high variance. $\Rightarrow$ Tend to **overfitting**.
- As we increase k we reduce the variance, at the cost of higher bias. $\Rightarrow$ Tend to **underfitting**.

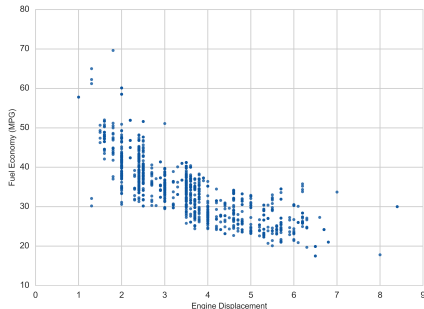
Illustration



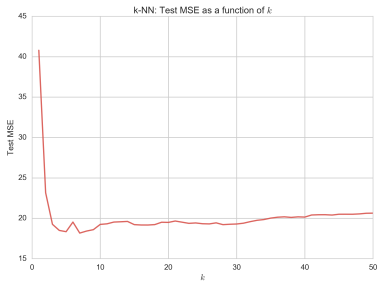
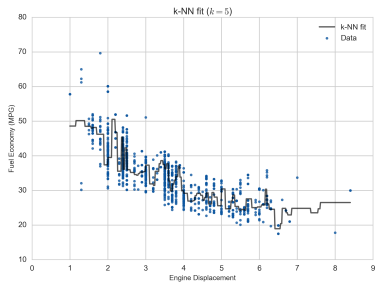
Play the animation to see how the estimates change as we vary k .

KNN illustration: predicting fuel economy

Data extracted from the fueleconomy.gov. We are interested in predicting highway miles per gallon (MPG) of a vehicle by a predictor “engine displacement”, which measures the size of cylinders.



KNN illustration: predicting fuel economy



Curse of dimensionality

- The KNN method is subject to a **curse of dimensionality**: it breaks down with high-dimensional inputs (high p).
- The reason is that as we increase the number of predictors, it becomes exponentially more difficult to find training observations are reasonably local to the prediction point $\mathbf{x}$.
- The curse of dimensionality is a prevalent problem with nonparametric methods.

Computational considerations

- The KNN method is a memory intensive method. It requires us to keep the entire training sample in the memory for computing predictions.
- Generating predictions is computationally costly. For each new input point, we need to compute distances to all the training points, and sort these values. That contrasts with linear regression, where computing predictions is cheap.
- Sorting algorithms require a number of computations which is proportional to $N \log(N)$ on average, therefore the KNN method does not scale well to large datasets.

Comparison with linear regression

Comparison with linear regression

Linear regression: $\hat{f}(\mathbf{x}) = \hat{\beta}_0 + \sum_{j=1}^p \hat{\beta}_j x_j$

KNN: $\hat{f}(\mathbf{x}) = \frac{1}{k} \sum_{\mathbf{x}_i \in N_k(\mathbf{x}, \mathcal{D})} y_i$

Comparison with linear regression

- The linear regression and KNN methods represent two opposite approaches to supervised learning.
- Linear regression assumes a linear form for the regression function $f(\mathbf{x})$. This assumption leads to stable predictions $\hat{f}(\mathbf{x})$, but the model can be highly inaccurate (high bias) if the assumption of linearity in the parameters is incorrect.
- KNN makes no structural assumptions about $f(\mathbf{x})$, leading to highly flexible predictions (low bias). But its predictions can be very unstable (high variance), since only a few training observations contribute to each prediction.

Parametric or nonparametric?

- As always, the choice of method should be based on estimating the test performance with the available data.
- In general, we can say that a parametric model will outperform the nonparametric approach if the parametric form assumed for the regression function is close to the true form of f .

Summary

Summary

- The KNN algorithm is a highly flexible method that does not explicitly assume a form for the regression function $f(X)$.
- A smaller value of k provides the more flexible model, with low bias but high variance.
- A larger value of k provides smoother estimates, at the cost of a less flexible approximation, with low variance but high bias.

Summary

However, the KNN method has disadvantages that we need to keep in mind:

- The estimate of the regression function can be very unstable, since it is the average of only a few points. This is the price that we pay for flexibility.
- Curse of dimensionality.
- The predictive performance is sensitive to noisy or irrelevant predictors.
- Generating predictions is computationally expensive.

Review questions

- How does the KNN method compute predictions?
- What is the hyperparameter?
- What is the Euclidean distance between two points?
- What is the curse of dimensionality?
- What are the advantages and disadvantages of the KNN method?